

Ejercicio 7

Para cada uno de los algoritmos presentados:

- Expresar en función de n el tiempo de ejecución.
- Establecer el orden de dicha función usando notación Big-Oh.

1)

```
for(int i = 0; i < n + 100; ++i) {  
    for(int j = 0; j < i * n ; ++j)  
        sum = sum + j;  
    for(int k = 0; k < n + n + n; ++k)  
        c[k] = c[k] + sum;  
}
```

a. En el código tenemos una iteración con otras dos iteraciones dentro. Cada for va a representar una sumatoria.

- Segundo for
Comienza desde $j = 0$ hasta $i \cdot n - 1$. Esto es porque el límite superior escrito en el for es estrictamente menor a $i \cdot n$ y el índice aumenta en uno en cada iteración. Hay dos opciones:
 - Tomar el límite inferior desde $j = 0$ hasta $i \cdot n - 1$.
 - Tomar el límite inferior desde $j = 1$ hasta $i \cdot n$.

Voy a elegir la segunda opción. La sumatoria me queda:

$$\sum_{k=1}^{i \cdot n} c_1$$

- Tercer for
Comienza desde $k = 0$ hasta $n + n + n - 1$, que es igual a $3n - 1$. Esto es porque el límite superior escrito en el for es estrictamente menor a $3n$ y el índice aumenta en uno en cada iteración. Hay dos opciones:
 - Tomar el límite inferior desde $k = 0$ hasta $3n - 1$.

- Tomar el límite inferior desde $k = 1$ hasta $3n$.

Voy a elegir la segunda opción. La sumatoria me queda

$$\sum_{k=1}^{3n} c_2$$

- Primer for

Comienza desde $i = 0$ hasta $n+99$. Esto es porque el límite superior escrito en el for es estrictamente menor a $n + 100$ y el índice aumenta en uno en cada iteración. Hay dos opciones:

- Tomar el límite inferior desde $i = 0$ hasta $n+99$.
- Tomar el límite inferior desde $i = 1$ hasta $n+100$.

Voy a elegir la segunda opción. Esta sumatoria está compuesta por la suma de las anteriores.

$$\sum_{i=1}^{n+100} \left(\sum_{k=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right)$$

Que en este caso, también corresponde a la "forma inicial" de $T(n)$.

Podemos armar una expresión de $T(n)$.

$$T(n) = \sum_{i=1}^{n+100} \left(\sum_{k=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n .

Propiedades que voy a usar:

- $\sum_{i=1}^n c = (n - 1 + 1) \times c = n \times c$
- $\sum_{i=1}^n c \cdot a_i = c \times \sum_{i=1}^n a_i$
- $\sum_{i=1}^n a_i + b_i = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Entonces,

$$\begin{aligned}
T(n) &= \sum_{i=1}^{n+100} \left(\sum_{j=1}^{i \cdot n} c_1 + \sum_{k=1}^{3n} c_2 \right) = \\
&= \sum_{i=1}^{n+100} (c_1 i \cdot n + c_2 3 \cdot n) = \\
&= \sum_{i=1}^{n+100} c_1 i n + \sum_{i=1}^{n+100} c_2 3n = \\
&= c_1 n \sum_{i=1}^{n+100} i + c_2 3n \sum_{i=1}^{n+100} 1 = \\
&= c_1 n \left(\frac{(n+100)(n+101)}{2} \right) + c_2 3n(n+100) = \\
&= \frac{c_1}{2} n(n^2 + 201n + 201) + c_2 3n^2 + c_2 300n \\
&= \frac{c_1}{2} n^3 + \frac{c_1}{2} 201n^2 + \frac{c_1}{2} 201n + c_2 3n^2 + c_2 300n = \\
&= \frac{c_1}{2} n^3 + \left(\frac{c_1}{2} 201 + c_2 3 \right) n^2 + \left(\frac{c_1}{2} 201 + c_2 300 \right) n
\end{aligned}$$

Llamando

- $c_3 = \frac{c_1}{2}$
- $c_4 = \frac{c_1}{2} 201 + 3c_2$
- $c_5 = 201 \frac{c_1}{2} + 300c_2$

Tenemos el tiempo de ejecución T en función de n .

$$T(n) = c_3 n^3 + c_4 n^2 + c_5 n$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^3 crece más rápido que n^2 y que n . Se tiene que demostrar que

$$T(n) = c_3 n^3 + c_4 n^2 + c_5 n \leq O(n^3)$$

Para esto, tenemos que encontrar una constante $k > 0$ y un n_0 tales que

$$c_3 n^3 + c_4 n^2 + c_5 n \leq k n^3 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: $c_3 n^3$
Sabemos que $n^3 \leq n^3$ Si multiplicamos a ambos lados por c_3 ,

$$c_3 n^3 \leq c_3 n^3$$

Encontramos un valor $k_1 = c_3$. Ahora, si probamos con $n_0 = 0$.

$$c_3 \cdot 0^3 \leq c_3 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 0$, para $n_0 \geq 0$.

- Segundo término: $c_4 n^2$
Sabemos que $n^2 \leq n^3$ Si multiplicamos a ambos lados por c_4 ,

$$c_4 n^2 \leq c_4 n^3$$

Encontramos un valor $k_2 = c_4$. Ahora, si probamos con $n_0 = 0$.

$$c_4 \cdot 0^2 \leq c_4 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: $c_5 n$
Sabemos que $n \leq n^3$ Si multiplicamos a ambos lados por c_5 ,

$$c_5 n \leq c_5 n^3$$

Encontramos un valor $k_3 = c_5$. Ahora, si probamos con $n_0 = 0$.

$$c_5 \cdot 0 \leq c_5 \cdot 0^3$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_5$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3 n^3 + c_4 n^2 + c_5 n \leq k_1 n^3 + k_2 n^3 + k_3 n^3$$

$$T(n) \leq (k_1 + k_2 + k_3) n^3$$

$$T(n) \leq (c_3 + c_4 + c_5) n^3$$

Renombrando $c = c_3 + c_4 + c_5$

$$T(n) \leq c n^3$$

El n_0 más restrictivo es $n_0 = 0$ porque cumple con todos los términos para $n_0 \geq 0$. Por lo tanto,

$$T(n) \leq O(n^3), \text{ con } c = c_3 + c_4 + c_5 \text{ para todo } n \geq n_0, \text{ con } n_0 = 0.$$

2)

```

int i,j;
int x = 1;
for (i = 0; i <= n^2; i=i+2)
for (j = n; j >= 1; j-= n/4)
    x++;

```

a. Primero tenemos dos operaciones constantes, las líneas

```

int i,j;
int x = 1;

```

y dos iteraciones anidadas que corresponden a dos sumatorias, una externa y otra interna. Vamos a analizar como va cambiando el índice de cada una.

- Sumatoria interna.

Este for empieza con $j = n$ y en cada "vuelta" se le resta $\frac{n}{4}$ al valor de j hasta que sea 1. Entonces

1. $j = n$ (primera vuelta)
2. $j = n - \frac{n}{4}$ (segunda vuelta)
3. $j = n - \frac{n}{4} - \frac{n}{4} = n - 2\frac{n}{4}$ (tercera vuelta)
4. $j = n - 2\frac{n}{4} - \frac{n}{4} = n - 3\frac{n}{4}$ (cuarta vuelta)
5. $j = n - 3\frac{n}{4} - \frac{n}{4} = n - 4\frac{n}{4} = n - n = 0$ (Esta vez ya no se ejecuta)

El for interno se ejecuta 4 veces, una cantidad constante. La sumatoria nos queda

$$\sum_{j=1}^4 c_2$$

- Sumatoria externa

Este for deja de ejecutarse cuando $i = n^2$ y aumenta en $i + 2$ en cada iteración. Si probamos con distintos valores de n .

- Con $n = 3$
 1. $i = 0$
 2. $i = 2$
 3. $i = 6$
 4. $i = 8$
- Con $n = 4$
 1. $i = 0$
 2. $i = 2$

$$3. i = 4$$

$$4. i = 8$$

$$5. i = 10$$

$$6. i = 12$$

$$7. i = 14$$

$$8. i = 16$$

El for externo se ejecuta $\frac{n^2}{2}$ veces.

$$\sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right)$$

Podemos armar una expresión de $T(n)$.

$$T(n) = c_1 + \sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n . Propiedades a usar:

- $\sum_{k=m}^n c = c(n - m + 1)$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=0}^{\frac{n^2}{2}} \left(\sum_{j=1}^4 c_2 \right) \\ &= c_1 + \sum_{i=0}^{\frac{n^2}{2}} 4c_2 = \\ &= c_1 + 4c_2 \left(\frac{n^2}{2} - 0 + 1 \right) = \\ &= c_1 + 4c_2 + 2c_2n^2 \end{aligned}$$

Si renombramos:

- $c_3 = c_1 + 4c_2$
- $c_4 = 2c_2$

Tenemos el tiempo de ejecución T en función de n .

$$\boxed{T(n) = c_3 + c_4n^2}$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^2 crece más rápido que n . Se tiene que demostrar que

$$T(n) = c_3 + c_4 n^2 \leq O(n^2)$$

Para esto, tenemos que encontrar una constante $k \neq 0$ y un n_0 tales que

$$c_3 + c_4 n^2 \leq k n^2 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: c_3
Sabemos que $1 \leq n^2$. Si multiplicamos a cada lado por c_3 .

$$c_3 \leq c_3 n^2$$

Encontramos entonces un valor k_1 que acota al término, $k_1 = c_3$. Si probamos con $n_0 = 0$, la desigualdad no se cumple porque $c_3 \geq c_3(0)^2$. Para $n_0 = 1$,

$$c_3 \leq c_3 \cdot 1^2 c_3 \leq c_3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término: $c_4 n^2$
Sabemos que $n^2 \leq n^2$. Si multiplicamos a ambos lados por c_4 ,

$$c_4 n^2 \leq c_4 n^2$$

Encontramos un valor para $k_2 = c_4$. Si tomamos $n_0 = 0$,

$$c_4 \cdot 0^2 \leq c_4 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3 + c_4 n^2 \leq k_1 n^2 + k_2 n^2$$

$$T(n) \leq (k_1 + k_2) \cdot n^2$$

$$T(n) \leq (c_3 + c_4) \cdot n^2$$

Renombrando $c = c_3 + c_4$

$$T(n) \leq c n^2$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^2), \text{ con } c = c_3 + c_4 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$

3)

```

public static void calcular(int n) {
    int i, j, r = 0;
    for ( i = 1; i < n; i= i+2)
        for (j = 1; j <= i; j++ )
            r = r + 1;

    return r;
}

```

a. En este algoritmo tenemos, en la primera línea, operaciones constante que vamos a representar con c_1 y dos for anidados que van a ser traducidas como una sumatoria con otra adentro.

- La sumatoria interna (el segundo for), se va ejecutar i veces (teniendo en cuenta también, que el índice aumenta de uno en uno), realizándose una operación constante. Vamos a tomar los límites de la sumatoria desde $j = 1$ hasta i .

$$\sum_{j=1}^i c_2$$

- La sumatoria externa Se ejecuta desde n hasta el valor anterior a n . El índice cambia sumándose en 2 luego de cada iteración. Veamos como cambia el índice en cada iteración:

1. $i = 1$
2. $i = 1 + 2$
3. $i = 1 + 2 + 2 = 1 + 2 \cdot 2$
4. $i = 1 + 2 \cdot 2 + 2 = 1 + 2 \cdot 3$

$\vdots$

$k.$ $i = 1 + 2 \cdot (k - 1)$ Paso general

Como el for se ejecuta hasta que $i = n$, despejamos n del paso general para saber cuándo el índice llega al valor de $n - 1$. Agarré $n - 1$ porque nunca la condición de corte es $i < n$, no $i \leq n$.

$$n - 1 = 1 + 2(k - 1)$$

$$n - 1 = 2k - 2$$

$$\frac{n+1}{2} = k \quad \text{Este es el límite superior de la sumatoria}$$

Entonces, la sumatoria del primer for queda,

$$\sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right)$$

Con esto, se puede armar una expresión de $T(n)$.

$$T(n) = c_1 + \sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right)$$

Ahora, tenemos que "deshacernos" de las sumatorias y hacer que todo quede en función de n . Propiedades a usar:

- $\sum_{i=1}^n c = (n - 1 + 1) \times c = n \times c$
- $\sum_{i=1}^n c \cdot a_i = c \times \sum_{i=1}^n a_i$
- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

Entonces,

$$\begin{aligned} T(n) &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} \left(\sum_{j=1}^i c_2 \right) \\ &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} (c_2 \cdot (i - 1 + 1)) \\ &= c_1 + \sum_{i=1}^{\frac{n+1}{2}} c_2 i \\ &= c_1 + c_2 \sum_{i=1}^{\frac{n+1}{2}} i \\ &= c_1 + c_2 \cdot \left(\frac{\frac{n+1}{2} \cdot (\frac{n+1}{2} + 1)}{2} \right) \\ &= c_1 + c_2 \cdot \left(\left(\frac{n+1}{2} \right) \left(\frac{n+3}{4} \right) \right) \\ &= c_1 + c_2 \frac{1}{8} \cdot (n^2 + 4n + 3) \\ &= c_1 + \frac{3}{8}c_2 + \frac{1}{2}c_2n + \frac{1}{8}c_2n^2 \end{aligned}$$

Renombrando

- $c_3 = c_1 + \frac{3}{8}c_2$
- $c_4 = \frac{1}{2}c_2$
- $c_5 = \frac{1}{8}c_2$

Tenemos el tiempo de ejecución T en función de n .

$$\boxed{T(n) = c_3 + c_4n + c_5n^2}$$

b. Para establecer el orden de la función, hay que hacer la demostración. Podemos decir que n^2 crece más rápido que n . Se tiene que demostrar que

$$T(n) = c_3 + c_4n + c_5n^2 \leq O(n^2)$$

Para esto, tenemos que encontrar una constante $k \geq 0$ y un n_0 tales que

$$c_3 + c_4n + c_5n^2 \leq kn^2 \text{ para todo } n \geq n_0$$

Empezamos analizando término a término.

- Primer término: c_3
Sabemos que $1 \leq n^2$. Si multiplicamos a cada lado por c_3 .

$$c_3 \leq c_3n^2$$

Encontramos entonces un valor k_1 que acota al término, $k_1 = c_3$. Si probamos con $n_0 = 0$, la desigualdad no se cumple porque $c_3 \geq c_3(0)^2$. Para $n_0 = 1$,

$$c_3 \leq c_3 \cdot 1^2 c_3 \leq c_3$$

La desigualdad se cumple. El término se puede acotar con $k_1 = c_3$ y $n_0 = 1$, para $n_0 \geq 1$.

- Segundo término: c_4n
Sabemos que $n \leq n^2$. Si multiplicamos a ambos lados por c_4 ,

$$c_4n \leq c_4n^2$$

Encontramos un valor para $k_2 = c_4$. Si tomamos $n_0 = 0$,

$$c_4 \cdot 0 \leq c_4 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_2 = c_4$ y $n_0 = 0$, para $n_0 \geq 0$.

- Tercer término: c_5n^2
Sabemos que $n^2 \leq n^2$. Si multiplicamos a ambos lados por c_5 ,

$$c_5n^2 \leq c_5n^2$$

Encontramos un valor para $k_3 = c_5$. Si tomamos $n_0 = 0$,

$$c_5 \cdot 0^2 \leq c_5 \cdot 0^2$$

La desigualdad se cumple. El término se puede acotar con $k_3 = c_5$ y $n_0 = 0$, para $n_0 \geq 0$.

Ahora, sumamos todos los resultados parciales que obtuvimos antes.

$$c_3 + c_4n + c_5n^2 \leq k_1n^2 + k_2n^2 + k_3n^2$$

$$T(n) \leq (k_1 + k_2 + k_3) \cdot n^2$$

$$T(n) \leq (c_3 + c_4 + c_5) \cdot n^2$$

$$\text{Renombrando } c = c_3 + c_4 + c_5$$

$$T(n) \leq cn^2$$

El n_0 más restrictivo es $n_0 = 1$ porque cumple con todos los términos para $n_0 \geq 1$. Por lo tanto,

$$T(n) \leq O(n^2), \text{ con } c = c_3 + c_4 + c_5 \text{ para todo } n \geq n_0, \text{ con } n_0 = 1.$$